

SHETH LUJ AND SIR MV COLLEGE

Artificial Intelligence

Practical No 7

Aim: Naive Bayes' Classifier

- Implement the Naive Bayes' algorithm for classification.
- Train a Naive Bayes' model using a given dataset and calculate class probabilities.
- Evaluate the accuracy of the model on test data and analyze the results.

Code:

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Importing the dataset
dataset = pd.read_csv('Daily Coffee Intake vs Sleep Duration.csv')

# Handling missing values in 'Age' just in case
if 'Age' in dataset.columns:
    dataset['Age'] = dataset['Age'].fillna(dataset['Age'].median())

# Selecting features (X) and target (y)
# 1. Include health and coffee metrics, y is Sleep_Quality
X = dataset[['Age', 'Coffee_Intake', 'Caffeine_mg', 'Sleep_Hours', 'BMI', 'Heart_Rate', 'Physical_Activity_Hours']].values
y = dataset['Sleep_Quality'].values

# Splitting the dataset into the training set and test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20, random_state = 0)

# Feature Scaling
# Since distance/variance matters, we scale the numeric features
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Training the Naive Bayes model on the training set
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)

# Predicting the test set results
y_pred = classifier.predict(X_test)
print('Predicted Test Results : ', y_pred)
print("\n")

# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix, accuracy_score
ac = accuracy_score(y_test, y_pred)
print('Model Accuracy : ', ac * 100, "%")
print("\n")
cm = confusion_matrix(y_test, y_pred)
print('Model Confusion Matrix : ')
print(cm)
```

Output:

```
[1]: print("\n")

# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix, accuracy_score
ac = accuracy_score(y_test, y_pred)
print('Model Accuracy : ', ac * 100, "%")
print("\n")
cm = confusion_matrix(y_test, y_pred)
print('Model Confusion Matrix : ')
print(cm)

Predicted Test Results : ['Good' 'Fair' 'Poor' ... 'Poor' 'Good' 'Good']
Model Accuracy : 96.05 %
Model Confusion Matrix :
[[ 236   0  36   0]
 [   0 387  21   0]
 [  16  14 319   0]
 [   0   0  1773]]

[2]: Start coding or generate with AI.

[3]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics
from sklearn.preprocessing import StandardScaler, LabelEncoder
import matplotlib.pyplot as plt

print('libraries imported successfully')

# 1. Load the dataset
df = pd.read_csv('Daily Coffee Intake vs Sleep Duration.csv')
print('Dataset loaded')

# 2. Data Cleaning
# If there are any missing values in the 'Age' column, we fill them with the median age.
if 'Age' in df.columns:
    df['Age'] = df['Age'].fillna(df['Age'].median())

# 3. Feature Selection and Target Initialization
# We select a set of health and coffee metrics for our predictors (X).
feature_cols = [
    'Age', 'Coffee_Intake', 'Caffeine_mg', 'Sleep_Hours',
    'BMI', 'Heart_Rate', 'Physical_Activity_Hours']
y_col = 'Sleep_Quality'
```

Omith Thilakan

T097

SHETH LUJ AND SIR MV COLLEGE

Artificial Intelligence

Code:

```
colab.research.google.com/drive/1J99bRSqdCnZmVTPv_jSmO3W7WS5_f8tV#scrollTo=YimnjWls72jZ

Practical No 3 Artificial Intelligence T097

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
import matplotlib.pyplot as plt

print("Libraries imported successfully!")

# 1. Load the dataset
df = pd.read_csv("Daily Coffee Intake vs Sleep Duration.csv")
print("Dataset loaded")

# 2. Data Cleaning
# If there are any missing values in the 'Age' column, we fill them with the median age.
df['Age'] = df['Age'].fillna(df['Age'].median())

# 3. Feature Selection and Target Initialization
# We select a mix of health and coffee metrics for our predictors (X).
feature_cols = [
    'Age', 'Coffee_Intake', 'Caffeine_mg', 'Sleep_Hours',
    'BMI', 'Heart_Rate', 'Physical_Activity_Hours'
]
X = df[feature_cols].copy()

# The target variable (y) is predicting Sleep_Quality.
# LabelEncoder converts our categorical text ('Good', 'Poor', etc.) into numerical format.
le = LabelEncoder()
y = le.fit_transform(df['Sleep_Quality'])

# 4. Encoding Categorical Features
# We loop through our predictor columns and convert any text into numbers (if applicable).
for col in X.columns:
    if X[col].dtype == 'object':
        X[col] = le.fit_transform(X[col].astype(str))

# 5. Data Transformation
std = StandardScaler()
X_scaled = std.fit_transform(X)
print("Transformed Data: ", X_scaled[2:10]) # Printing first 2 rows for brevity

test_size = 0.30
seed = 7

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=test_size, random_state=seed)

# 7. Model Initialization and Training
model = GaussianNB()
model.fit(X_train, y_train)
print("Model trained")

# 8. Making Predictions
predicted = model.predict(X_test)

# 9. Result Diagnosis
conf_matrix = metrics.confusion_matrix(y_test, predicted)
print("Confusion Matrix:\n", conf_matrix)

model_score = model.score(X_test, y_test)
print("Model Score (Accuracy):", model_score)

# 10. Probability Predictions for ROC
y_predictProb = model.predict_proba(X_test)

# 11. ROC Curve Calculation
from sklearn.metrics import auc, roc_curve

# Note: Because 'Sleep_Quality' has multiple classes, we binarize it against a specific encoded class (e.g., class 1) to plot the binary ROC curve.
fpr, tpr, thresholds = roc_curve(y_test == 1, y_predictProb[:, 1])
roc_auc = auc(fpr, tpr)
print("ROC AUC:", roc_auc)

# 12. Plot ROC Curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='navy', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC)')
plt.legend(loc='lower right')
plt.show()

Libraries imported successfully!
Dataset loaded
Transformed Data:
[[ 0.40297415  0.68328715  0.65113791  0.78688899  0.23378588  0.75156324
  1.65264693]
 [-0.17468457 -1.0407226 -1.04709124 -0.35697384 -1.02864595 -0.36831913
  0.35415402]]
Model trained: GaussianNB()
```

Omith Thilakan
T097

SHETH LUJ AND SIR MV COLLEGE

Artificial Intelligence

Output:

